

Facultad de Ingeniería – UNA

Carrera: Ingeniería Mecatrónica

Asignatura: Lenguajes de Programación Visual

Primer Examen Parcial Práctico (Modalidad: 24 Horas)

Docentes: Ing. Jorge Tillería e Ing. Marcos Benítez

1. Pautas y Reglas Generales

- **Modalidad:** Individual, formato *take-home* (entrega estricta en 24 horas continuas a partir de la publicación).
- **Ecosistema y Lenguaje: Totalmente libre.** Puedes implementarlo en el lenguaje y framework de tu preferencia (Python con FastAPI/Flask, Node.js/TypeScript con Express/Nest, Go, Rust, C#, Java, etc.).
- **Persistencia:** Elección libre del motor de base de datos (Relacional como SQLite/PostgreSQL o NoSQL basado en documentos como MongoDB).
- **Archivos Adjuntos Provistos:**
 1. `simulador_planta.exe` : Binario compilado del concentrador de planta (cliente de homologación automática).
 2. `documentacion_interconexion.pdf` : Especificación técnica formal elaborada por el área de instrumentación de planta con el detalle de las señales y tramas de campo.

2. Contexto de Ingeniería del Problema

En la planta industrial, múltiples nodos periféricos capturan variables de proceso y transmiten sus mediciones hacia un concentrador central (*Edge Gateway*).

La ingeniería de enlace ya fue implementada por el área de instrumentación: el concentrador traduce las señales de campo y las despacha vía HTTP a un servidor central. Todos los parámetros físicos, modelos de sensores y tablas de conversión se detallan en el documento adjunto (`documentacion_interconexion.pdf`).

Tu objetivo: Diseñar, estructurar e implementar el **Backend y la API REST** que reciba, persista, consulte y administre las entidades requeridas por dicho concentrador y por las

estaciones de supervisión.

3. Especificación Técnica de la API REST (CRUD Completo)

Tu servicio debe operar bajo el protocolo HTTP, responder con el encabezado `Content-Type: application/json` y resolver los contratos de datos detallados a continuación.

A. Módulo de Sensores (`/api/sensores`)

- **1. Alta de Sensor (`POST /api/sensores`):**

- **Cuerpo de la Petición (Request Body):**

```
{
  "nodo": "ESP32-Caldera-01",
  "tipo": "Temperatura",
  "modelo": "DS18B20",
  "unidad": "°C",
  "rango_max": 85.0
}
```

- **Respuesta Exitosa (`201 Created`):** Objeto persistido con su identificador generado (bajo `"id"` , `"sensor_id"` o `"_id"`):

```
{
  "id": 1,
  "nodo": "ESP32-Caldera-01",
  "tipo": "Temperatura",
  "modelo": "DS18B20",
  "unidad": "°C",
  "rango_max": 85.0
}
```

- **Respuesta de Error (`400 Bad Request`):** Estructura incompleta o tipos inválidos.

- **2. Listado de Sensores (`GET /api/sensores`):**

- **Respuesta Exitosa (`200 OK`):** Lista (`Array`) con los sensores registrados:

```
[
  {
    "id": 1,
    "nodo": "ESP32-Caldera-01",
    "tipo": "Temperatura",
    "modelo": "DS18B20",
    "unidad": "°C",
    "rango_max": 85.0
  }
]
```

- **3. Actualización de Parámetros (PUT /api/sensores/{sensor_id}):** *(Operación de administración: no ejecutada por el simulador pero requerida para completar el ciclo CRUD).*

- **Cuerpo de la Petición (Request Body):** Campos a redefinir (ej. recalibración de umbral o cambio de nodo):

```
{
  "nodo": "ESP32-Caldera-01",
  "tipo": "Temperatura",
  "modelo": "DS18B20",
  "unidad": "°C",
  "rango_max": 90.0
}
```

- **Respuesta Exitosa (200 OK):** Retorna el objeto actualizado.
- **Respuesta de Error (404 Not Found):** Si el `sensor_id` no existe en la base de datos.
- **4. Baja de Sensor (DELETE /api/sensores/{sensor_id}):** *(Operación de administración: eliminación lógica o física del sensor).*
 - **Respuesta Exitosa (200 OK o 204 No Content):** Confirma la remoción del registro.
 - **Respuesta de Error (404 Not Found):** Si el `sensor_id` no existe.

B. Módulo de Lecturas y Telemetría (/api/lecturas)

- **1. Ingesta de Lectura (POST /api/lecturas):**
 - **Cuerpo de la Petición (Request Body):**

```
{
  "sensor_id": 1,
  "valor": 68.42,
  "timestamp": "2026-09-17T12:00:00Z"
}
```

- **Respuesta Exitosa (201 Created):** Registro de la muestra confirmado.
- **Respuestas de Error:** 404 Not Found si el sensor_id no existe en el sistema; 400 Bad Request si los datos son inválidos o están ausentes.

- **2. Histórico de Mediciones (GET /api/lecturas):**

- **Parámetros Opcionales (Query Params):** ?sensor_id=1&limit=50
- **Respuesta Exitosa (200 OK):** Arreglo JSON (Array) con los registros históricos almacenados:

```
[
  {
    "id": 101,
    "sensor_id": 1,
    "valor": 68.42,
    "timestamp": "2026-09-17T12:00:00Z"
  }
]
```

- **3. Corrección de Registro (PUT /api/lecturas/{lectura_id}):** (Operación de administración: ajuste manual de una lectura mal calibrada).

- **Cuerpo de la Petición (Request Body):**

```
{
  "valor": 67.50
}
```

- **Respuesta Exitosa (200 OK):** Retorna la lectura modificada.
- **Respuesta de Error (404 Not Found):** Si la lectura no existe.

- **4. Depuración de Lectura (DELETE /api/lecturas/{lectura_id}):** (Operación de administración: purga de un dato espurio o corrupto).

- **Respuesta Exitosa (200 OK o 204 No Content):** Confirma la eliminación de la medición.
- **Respuesta de Error (404 Not Found):** Si la lectura no existe.

C. Módulo de Analítica (GET /api/lecturas/estadisticas/{sensor_id})

- **Método y Ruta:** GET /api/lecturas/estadisticas/{sensor_id}
- **Respuesta Exitosa (200 OK):** Resumen estadístico exacto de las mediciones registradas para ese sensor:

```
{
  "sensor_id": 1,
  "minimo": 62.15,
  "maximo": 77.80,
  "promedio": 69.45
}
```

- **Respuesta de Error (404 Not Found):** Si se consulta un `sensor_id` que no existe en el sistema.

4. Verificación y Homologación con `simulador_planta.exe`

El software concentrador provisto (`simulador_planta.exe`) audita automáticamente el flujo de producción de la planta:

1. Inicia tu servidor backend en tu entorno local.
2. Abre una terminal (CMD o PowerShell) y ejecuta el simulador indicando la dirección y puerto de tu API:

```
simulador_planta.exe http://localhost:8000
```

(Si corre en el puerto 8000 por defecto, puedes ejecutarlo con doble clic).

3. El simulador recreará la dinámica de planta ejecutando las fases de registro de sensores, emisión con inercia física, consulta general, cálculo de métricas y validación de códigos de error 404.
4. **Condición de homologación:** La consola debe finalizar mostrando:

🎉 === TODAS LAS PRUEBAS COMPLETADAS CORRECTAMENTE === sin reportar interrupciones ni fallos con ❌.

5. Criterios de Evaluación y Ponderación

Criterio	Ponderación	Aspectos Evaluados
Homologación de Planta y REST	35%	Ejecución sin fallos con <code>simulador_planta.exe</code> , USO semántico de verbos HTTP (<code>GET</code> , <code>POST</code> , <code>PUT</code> , <code>DELETE</code>), códigos de estado (<code>200</code> , <code>201</code> , <code>204</code> , <code>400</code> , <code>404</code>) y formato JSON.
Modelado y Persistencia	25%	Estructura de tablas/colecciones, relaciones entre entidades e integridad de datos ante inserciones, actualizaciones y borrados.
Arquitectura y Calidad de Código	20%	Modularidad, control de excepciones, validaciones de entrada y buenas prácticas según el lenguaje elegido.
Documentación de Despliegue	20%	Archivo <code>README.md</code> claro con instrucciones de instalación de dependencias y comando para levantar el servidor.

Formato de Entrega: Subir a la plataforma de la cátedra el enlace al repositorio (GitHub/GitLab) o el archivo comprimido `.zip` del proyecto con su código fuente y `README.md` (omitiendo directorios temporales o pesados de dependencias como `.venv`, `node_modules`, `.git`, binarios compilados, etc.).